

PAC-MAN GAME

A fully playable Pac-Man built from scratch in Java—no database, just Swing and pure OOP.

● Week 5 Complete	Java JDK 21	VS Code	Swing / JPanel	60 FPS Game Loop
-------------------	-------------	---------	----------------	------------------

WHAT WAS BUILT

Ghost Class	Movement & AI
✓ Ghost.java — base class for all ghosts ✓ position, speed, sprite, dx/dy fields ✓ chooseDirection() random movement logic ✓ isAligned() grid-snap check before turning ✓ canMove() wall collision check	✓ Avoids reversing direction when roaming ✓ Falls back to reversal if no other option ✓ Sprite loaded from assets/sprites/ ✓ Fallback red oval if sprite missing ✓ Ghost spawned at tile (9,9) in GamePanel

KEY TECHNIQUES

isAligned() — Grid-Locked Turning

The ghost only picks a new direction when isAligned() returns true — meaning its pixel position is within one speed unit of a tile center. This prevents the ghost from turning mid-tile and drifting diagonally, keeping movement crisp and collision detection accurate at all times.

chooseDirection() — No-Reversal Random AI

Each time the ghost reaches a tile center it shuffles all four directions randomly, skips any that would reverse its current heading, and picks the first passable one. If no forward options exist (dead end), it falls back and allows reversal. This gives natural-looking wandering without getting stuck.

Inheritance-Ready Base Class

Ghost.java uses protected fields and a protected chooseDirection() method so future subclasses (RedGhost, BlueGhost, etc.) can override just the targeting logic while reusing all movement, rendering, and collision code. This is the foundation for all four unique ghost AIs in Week 6.

SOURCE CODE

Ghost.java

java

```
import java.awt.*;
import java.awt.image.BufferedImage;
import javax.imageio.ImageIO;
import java.io.File;
import java.util.Random;

public class Ghost {
```

```

protected int row, col;
protected double x, y;
protected int dx, dy;
protected double speed = 1.5;
protected BufferedImage sprite;
protected Maze maze;
protected Random random = new Random();

public Ghost(Mazemaze,int    startRow, int startCol, String spritePath) {
    this.maze    = maze;
    this.row     =startRow;
    this.col     =startCol;
    this.x       = col * Maze.TILE_SIZE;
    this.y       = row * Maze.TILE_SIZE;
    this.dx      =0;
    this.dy      =0;

    try {
        sprite = ImageIO.read(new File(spritePath));
    } catch (Exception e) {
        System.out.println("Couldnot load sprite: " + spritePath);
    }
}

public void update() {
    if (isAligned()) chooseDirection();
    if (canMove(dx, dy)) {
        x += dx * speed;
        y += dy * speed;
    }else {
        x = col * Maze.TILE_SIZE;
        y = row * Maze.TILE_SIZE;
        dx =0;dy=0;
    }
    row = (int) Math.round(y / Maze.TILE_SIZE);
    col = (int) Math.round(x / Maze.TILE_SIZE);
}

protected void chooseDirection() {
    int[][] directions = {{0,-1},{0,1},{-1,0},{1,0}};
    java.util.List dirs = java.util.Arrays.asList(directions);
    java.util.Collections.shuffle(dirs, random);

    // Prefer non-reversing directions
    for (Object obj : dirs) {
        int[] d = (int[]) obj;
        if (d[0]==-dx&& d[1]==-dy) continue;
        if (canMove(d[0],d[1])){dx = d[0]; dy = d[1]; return; }
    }
    // Fall back: allow reversal
    for (Object obj : dirs) {
        int[] d = (int[]) obj;
        if (canMove(d[0],d[1])){dx = d[0]; dy = d[1]; return; }
    }
}

```

```

protected boolean canMove(int testDx, int testDy) {
    if (testDx == 0 && testDy == 0) return false;
    double nextX = x + testDx * speed;
    double nextY = y + testDy * speed;
    int checkRow = (int) Math.round(nextY / Maze.TILE_SIZE);
    int checkCol = (int) Math.round(nextX / Maze.TILE_SIZE);
    return !maze.isWall(checkRow, checkCol);
}

protected boolean isAligned() {
    double diffX = x - col * Maze.TILE_SIZE;
    double diffY = y - row * Maze.TILE_SIZE;
    return Math.abs(diffX) < speed && Math.abs(diffY) < speed;
}

public void draw(Graphics g) {
    if (sprite != null) {
        g.drawImage(sprite, (int)x, (int)y, Maze.TILE_SIZE, Maze.TILE_SIZE, null);
    } else {
        g.setColor(Color.RED);
        g.fillOval((int)x, (int)y, Maze.TILE_SIZE, Maze.TILE_SIZE);
    }
}

public int getRow() { return row; }
public int getCol() { return col; }
public double getX() { return x; }
public double getY() { return y; }
}

```

GamePanel.java

java

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;

public class GamePanel extends JPanel implements Runnable {
    private Thread gameThread;
    private Maze maze;
    private PacMan pacman;
    private ScoreManager scoreManager;
    private Ghost ghost;
    private boolean gameOver = false;
    private boolean won = false;

    public GamePanel() {
        maze = new Maze();
        pacman = new PacMan(maze, 1, 1);
        scoreManager = new ScoreManager();
        ghost = new Ghost(maze, 9, 9, "assets/sprites/RedGhost.png");

        setBackground(Color.BLACK);
        setPreferredSize(new Dimension(
            Maze.COLS * Maze.TILE_SIZE,
            Maze.ROWS * Maze.TILE_SIZE + 40));
        setFocusable(true);
    }
}

```

```

addKeyListener(new KeyAdapter() {
    @Override
    public void keyPressed(KeyEvent e) {
        switch (e.getKeyCode()) {
            case KeyEvent.VK_UP:      pacman.setDirection(0, -1); break;
            case KeyEvent.VK_DOWN:    pacman.setDirection(0,  1); break;
            case KeyEvent.VK_LEFT:    pacman.setDirection(-1, 0); break;
            case KeyEvent.VK_RIGHT:   pacman.setDirection(1,  0); break;
        }
    }
});

gameThread = new Thread(this);
gameThread.start();
}

@Override
public void run() {
    while (true) {
        if (!gameOver && !won) {
            pacman.update();
            ghost.update();

            int eaten = pacman.eatDot();
            if (eaten == 2) scoreManager.addDot();
            if (eaten == 3) scoreManager.addPowerPellet();

            if (maze.countDots() == 0) won = true;
        }
        repaint();
        try { Thread.sleep(16); }
        catch (InterruptedException e) { e.printStackTrace(); }
    }
}

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g;
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

    g2.setFont(new Font("Arial", Font.BOLD, 16));
    g2.setColor(Color.WHITE);
    g2.drawString("SCORE", 10, 18);
    g2.setColor(Color.YELLOW);
    g2.drawString(String.valueOf(scoreManager.getScore()), 10, 34);

    g2.setColor(Color.WHITE);
    g2.drawString("LIVES", 180, 18);
    for (inti=0; i<scoreManager.getLives(); i++) {
        g2.setColor(Color.YELLOW);
        g2.fillOval(180 + i * 22, 22, 14, 14);
    }

    g2.setColor(Color.WHITE);

```

```

        g2.setFont(new Font("Arial", Font.BOLD, 16));
        g2.drawString("PAC-MAN", 85, 25);

        Graphics2D g2maze = (Graphics2D) g2.create();
        g2maze.translate(0, 40);
        maze.draw(g2maze);
        pacman.draw(g2maze);
        ghost.draw(g2maze);
        g2maze.dispose();

        if (won)            drawOverlay(g2, "YOU WIN!", Color.YELLOW);
        else if (gameOver) drawOverlay(g2, "GAME OVER", Color.RED);
    }

    private void drawOverlay(Graphics2D g2, String message, Color color) {
        g2.setColor(new Color(0, 0, 0, 160));
        g2.fillRect(0, 0, getWidth(), getHeight());
        g2.setFont(new Font("Arial", Font.BOLD, 36));
        g2.setColor(color);
        FontMetrics fm = g2.getFontMetrics();
        int x = (getWidth() - fm.stringWidth(message)) / 2;
        int y = getHeight() / 2;
        g2.drawString(message, x, y);
    }
}

```

App.java

java

```

public class App {
    public static void main(String[] args) {
        GameWindow window = new GameWindow();
        window.setVisible(true);
    }
}

```

GameWindow.java

java

```

import javax.swing.JFrame;

public class GameWindow extends JFrame {
    public GameWindow() {
        setTitle("Pac-Man Game");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setResizable(false);
        GamePanel panel = new GamePanel();
        add(panel);
        pack();
        setLocationRelativeTo(null);
    }
}

```

10-WEEK PROGRESS

Week 1 — Setup & Window



Week 2 — Maze Design



Week 3 — Pac-Man Character	✓
----------------------------	---

Week 4 — Dot Collection & Scoring	✓
-----------------------------------	---

Week 5 — Ghost Base Class	✓
---------------------------	---

Weeks 6–7 — All Ghosts + AI	0%
-----------------------------	----

Weeks 8–10 — Polish + Ship	0%
----------------------------	----

UP NEXT



All Four Custom Ghosts

Create RedGhost, YellowGhost, BlueGhost and BrownGhost — each with a unique target-tile AI behaviour and its own sprite.

RedGhost.java · YellowGhost.java · BlueGhost.java · BrownGhost.java · target-tile AI